CPU, MEMORY AND I/O RESOURCE OPTIMIZATION

A PROJECT REPORT

Submitted by

REGNO	NAME
111624102215	**SURENDAR HARIPRASAD**
111624102219	**E THAMILNIDHI**
111624102217	**TAMIL MANI B**
111624102222	**THARUN**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

R. M. K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

PUDUVOYAL



OCTOBER 2026

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY
(An Autonomous Institution)
R.S.M. Nagar, PUDUVOYAL-601 206
Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

BONAFIDE CERTIFICATE

This is to certify that **SURENDAR HARIPRASAD, E THAMILNIDHI, TAMIL MANI B and THARUN**, bearing **Register Numbers 111624102215, 111624102219, 111624102217 and 111624102222** respectively, of **3rd Year, 5th Semester, Department of Computer Science and Engineering**, have successfully completed the mini project entitled **"CPU, Memory and I/O Resource Optimization"** during the academic year **2026–2027** under my guidance and supervision.

The work submitted in this report is a bonafide record of the work carried out by the student/team as part of the academic requirements of the course.

Name of the Student **Project Guide**

SURENDAR HARIPRASAD

E THAMILNIDHI

TAMIL MANI B

THARUN

ABSTRACT

An operating system keeps three resources busy: the processor, the memory and the input/output devices. For each of them it must choose a policy: which process runs next, which page leaves memory when a new one arrives, and which disk request the arm serves first. Textbooks teach these policies one at a time, on a single fixed example, so a student rarely sees how they behave on a workload of their own, or why the best choice changes from one workload to another. This mini project builds a simulator and an optimizer for the three resources, using only the Python standard library. The CPU module implements FCFS, SJF, SRTF, priority scheduling with and without pre-emption, and round robin; the memory module implements FIFO, LRU, OPT and CLOCK page replacement; the I/O module implements FCFS, SSTF, SCAN, C-SCAN, LOOK and C-LOOK disk scheduling. The optimizer runs every algorithm on the same workload, ranks them by the metric that matters (waiting time, page faults, head movement), names the best one with the reason, and tunes parameters such as the round-robin quantum and the number of frames. The simulators reproduce the textbook results exactly: 17, 3 and 5.67 time units of average waiting for FCFS, SJF and round robin; 15, 12 and 9 page faults for FIFO, LRU and OPT; and 640, 236 and 208 cylinders of head movement for FCFS, SSTF and LOOK. On 500 seeded random workloads SRTF cut the average waiting time from 18.15 to 7.69 time units against FCFS and was never beaten. In one combined scenario, the recommended choices reduced waiting time by 63%, page faults by 13% and head movement by 79% against FCFS, FIFO and FCFS. A command-line tool and a browser interface with Gantt charts, frame tables and head-path plots are provided, and 29 automated tests pass.

TABLE OF CONTENTS

S.No	Section	Page No.
1	Introduction	1
2	Objectives	3
3	Tools and Technologies	5
4	Architecture Diagram	6
5	Methodology	8
6	Results	13
7	Conclusion	23
8	Reference	25

1. INTRODUCTION

1.1 Background

A computer runs many programs at once on a small number of physical resources. The operating system is the program that shares those resources, and three of them decide most of what a user experiences: the processor (CPU), the main memory and the input/output devices, of which the disk is the slowest and the most important. For each resource the operating system has to take a decision again and again. The CPU scheduler decides which ready process runs next. The virtual-memory manager decides which page to remove from memory when a new page must be loaded. The disk scheduler decides which of the waiting requests the disk arm should serve first. The textbooks of Silberschatz, Galvin and Gagne [1], Tanenbaum and Bos [2] and Stallings [3] describe a family of algorithms for each decision.

The algorithms have different strengths. Shortest-job-first scheduling gives the lowest average waiting time but needs to know how long a job will run, and it can starve long jobs. Round robin gives every process a quick first response but pays for it in context switches. LRU page replacement works well when programs reuse recent pages, and FIFO can behave in the surprising way known as Belady's anomaly [5], where giving a program more memory makes it fault more often. Shortest-seek-time-first disk scheduling moves the arm the least, yet requests far from the head can wait for a long time. None of these algorithms is best in every situation, so *optimising* the use of a resource means choosing the algorithm, and tuning its parameters, for the workload at hand.

Table 1.1: The three resources, the decision taken for each and the algorithms studied

Resource	Decision made by the operating system	Algorithms in this project	Main measure
CPU	Which ready process runs next, and for how long	FCFS, SJF, SRTF, PRIORITY, PRIORITY-P, RR	Waiting, turnaround and response time; context switches
Memory	Which page to replace when all frames are full	FIFO, LRU, OPT, CLOCK	Number of page faults
I/O (disk)	Which pending request the disk arm serves next	FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK	Total head movement in cylinders

1.2 Problem Statement

In a course the algorithms are met one by one, each on a single example that has been chosen so that its answer is a round number. A student can follow the example, but cannot

easily answer the questions that matter in practice: how do the algorithms compare on a workload I choose? How much better is the best one than the simplest one? When does the ranking change? What is a good value for the time quantum, or for the number of frames? Existing tools tend to cover only one resource, to show only one algorithm at a time, or to stop at drawing a chart without saying which algorithm should be used.

The problem addressed here is to build one tool that simulates the main algorithms for the CPU, the memory and the disk, runs them side by side on the same workload, measures them with the standard metrics, and recommends the best choice with a reason, while reproducing the textbook results exactly so that its answers can be trusted.

1.3 Motivation

Three observations motivated the project. First, the gap between a simple policy and a good one is large and easy to measure: on the same process set the average waiting time can differ by a factor of five between FCFS and SJF (Section 6.2). Second, several classical results are properties that can be tested rather than only proved, such as the optimality of SRTF for waiting time, the absence of Belady's anomaly in LRU and OPT, and the fact that OPT is a lower bound for every replacement rule. A simulator allows each of them to be checked on thousands of generated workloads. Third, parameters such as the round-robin quantum have no universal best value; the right value depends on the lengths of the jobs, and a tool that sweeps the parameter makes that visible.

1.4 Scope

The tool is a simulator, not a modification of a real kernel. Time is counted in whole units. The CPU model has one processor, processes whose bursts are known in advance, and no I/O inside a process; context switches are counted but cost no time unless a cost is given to the tuning function. The memory model is one reference string running against a fixed number of frames. The disk model is a single arm on a disk of a given number of cylinders, with all requests present at the start; seek time is taken to be proportional to the distance moved and rotational delay is ignored. Multilevel feedback queues, multiple processors, working-set page allocation among several processes, and disk request arrival over time are outside the scope and are listed in Section 7.2.

2. OBJECTIVES

The main objective is to design and implement a simulator and optimizer for the CPU, memory and I/O resources of an operating system, and to evaluate the classical algorithms on generated workloads. The specific objectives are:

- To simulate six CPU scheduling algorithms and report waiting, turnaround and response time, utilisation, throughput and context switches.
- To simulate four page-replacement algorithms and report page faults, hits and fault rate, including the frame contents after every reference.
- To simulate six disk-scheduling algorithms and report the path of the arm and the total head movement.
- To provide an optimizer that compares the algorithms on the same workload, picks the best, explains the choice and states the saving against the simplest policy.
- To tune the two main parameters, the round-robin time quantum and the number of frames.
- To verify the simulators against the worked examples of the textbooks and against theoretical properties, using automated tests.
- To offer a command-line interface and a browser interface with Gantt charts, frame tables and head-path plots.

Table 2.1: Functional requirements

ID	Requirement
FR1	Simulate CPU scheduling with FCFS, SJF, SRTF, PRIORITY, PRIORITY-P and RR for a list of processes with arrival time, burst and priority
FR2	Simulate page replacement with FIFO, LRU, OPT and CLOCK for a reference string and a number of frames
FR3	Simulate disk scheduling with FCFS, SSTF, SCAN, C-SCAN, LOOK and C-LOOK for a request queue, head position and direction
FR4	Compare all algorithms of a resource on one workload and recommend the best with a reason
FR5	Show a Gantt chart, the frame table after each reference and the path of the head
FR6	Sweep the time quantum and the number of frames and report the best value
FR7	Report the combined saving of the recommended choices in one scenario
FR8	Reject wrong input (empty workload, zero burst, request outside the disk) with a clear message

Table 2.2: Non-functional requirements

Requirement	How it is met
Correctness	Textbook examples reproduced exactly; theoretical properties checked on generated workloads
Repeatability	Every generated workload comes from a fixed random seed
Portability	Python standard library only; nothing to install
Speed	A workload of 200 processes is compared across all six algorithms in a fraction of a second
Clarity	Each algorithm is a short function that follows its textbook definition

3. TOOLS AND TECHNOLOGIES

The whole project is written in Python so that every algorithm reads like its description in the textbook, and it uses no third-party package. Table 3.1 lists what was used.

Table 3.1: Tools and technologies used

Item	Tool	Purpose
Language	Python 3.10 or newer (developed and tested on 3.13)	Simulators, optimizer, interfaces
Libraries	Standard library only: `dataclasses`, `random`, `http.server`, `json`, `unittest`	Seeded generators, server, tests
Browser interface	One HTML file with CSS and plain JavaScript	Three tabs, tables, Gantt chart, frame table, head path in SVG
Server	`http.server.ThreadingHTTPServer`	JSON endpoints used by the page
Testing	`unittest`	29 automated tests
Charts for this report	matplotlib	Figures 6.1 to 6.7 (not needed to run the tool)
Editor and shell	Visual Studio Code, PowerShell or any terminal	Development

3.1 Running the project

The project is a folder named `resopt`. No installation step is needed. From inside the folder:

```
python -m unittest discover -s tests      # the 29 automated tests
python -m resopt.cli demo                 # textbook examples, all three
resources
python -m resopt.cli cpu --procs P1:0:24,P2:0:3,P3:0:3 --quantum 4
python -m resopt.cli memory --refs 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 --
frames 3
python -m resopt.cli disk --requests 98,183,37,122,14,124,65,67 --head 53
python -m resopt.server 8000              # browser interface at
http://127.0.0.1:8000
python bench.py                           # the experiments of Chapter 6
```

4. ARCHITECTURE DIAGRAM

The tool has four layers (Figure 4.1). The presentation layer is the command line, the browser page and the small HTTP server behind the page. The optimizer layer compares algorithms, tunes parameters and combines the three resources. The simulators are three independent modules, one per resource, and each has the same shape: a function that takes a workload and the name of an algorithm and returns a result with the full trace and the metrics. At the bottom, the workload generators produce seeded random inputs. A layer calls only the layer below it, so the simulators can be tested without any interface.

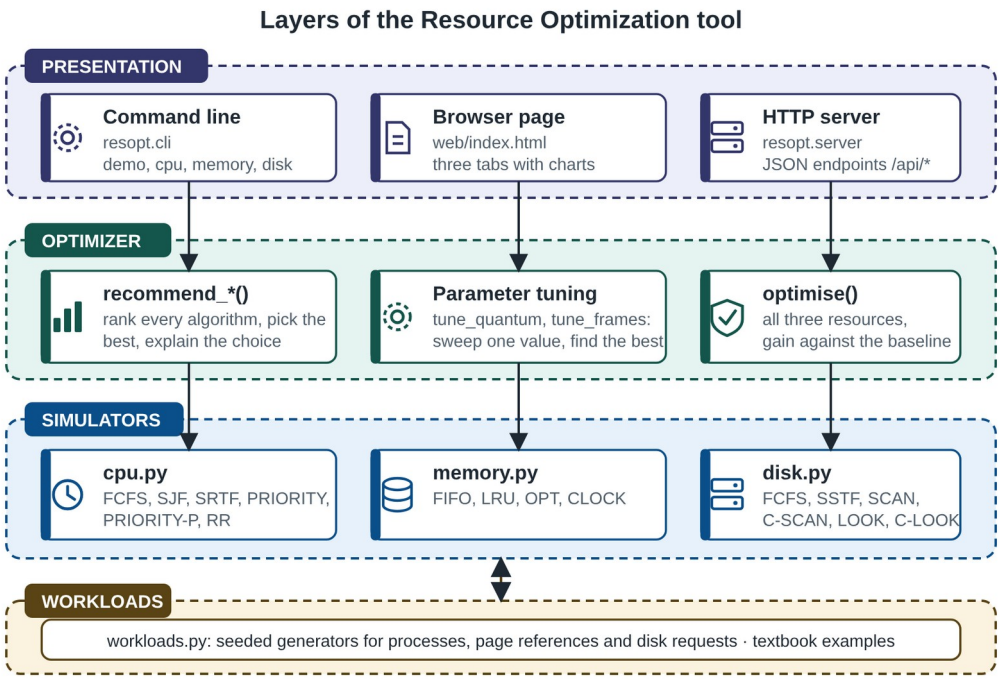


Figure 4.1: Layers of the Resource Optimization tool

Table 4.1: Module responsibilities

File	Lines	Responsibility
`resopt/cpu.py`	118	Six CPU scheduling algorithms, metrics, Gantt chart
`resopt/memory.py`	54	FIFO, LRU, OPT and CLOCK page replacement with a step-by-step trace
`resopt/disk.py`	53	Six disk-arm scheduling algorithms and head path
`resopt/optimizer.py`	69	Ranking, recommendation, quantum and frame tuning, combined scenario
`resopt/workloads.py`	41	Seeded generators of processes, reference strings and disk requests
`resopt/cli.py`, `resopt/server.py`	116	Command line and HTTP server
`web/index.html`	128	Browser page
`tests/test_resopt.py`	185	29 automated tests
`bench.py`	93	Experiments of Chapter 6

4.1 From a workload to a recommendation

Figure 4.2 shows what happens for one request. The workload is passed to every algorithm of the resource it belongs to, so the comparison is always like for like. Each simulator returns the metrics of its algorithm. The optimizer ranks them by the relevant metric (average waiting time for the CPU, page faults for memory, head movement for the disk), picks the best, and compares it with the baseline: FCFS for the CPU, FIFO for memory and FCFS for the disk. The result is the recommendation, the percentage saved, and a sentence that explains why the algorithm wins and what its drawback is. For memory the recommendation never names OPT, because OPT needs to know the future and is shown only as the lower bound that no real rule can beat.

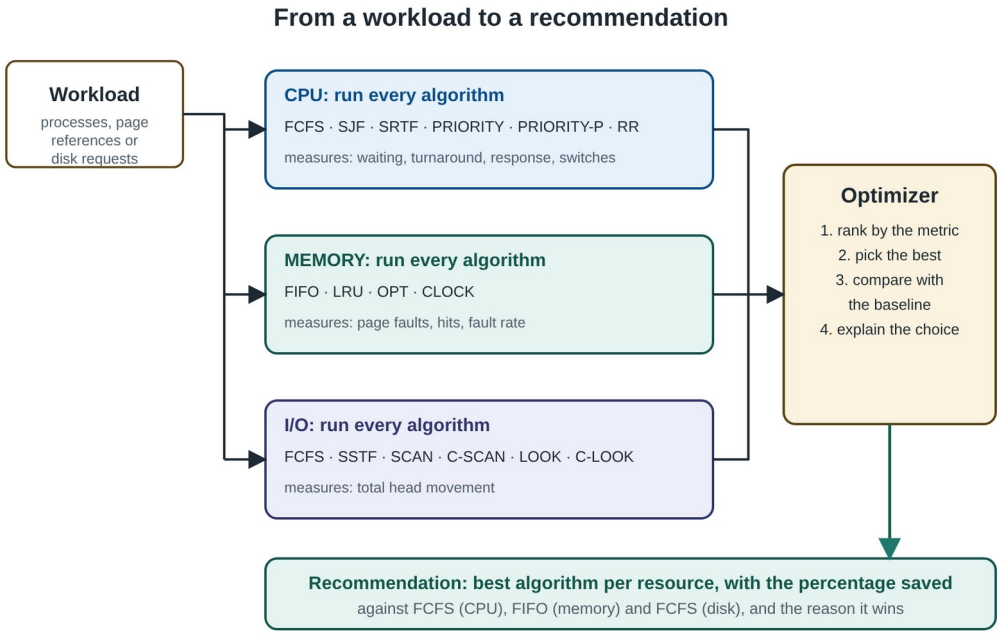


Figure 4.2: From a workload to a recommendation

5. METHODOLOGY

5.1 Overall approach

The method has three parts. First, each algorithm is implemented as a simulator that follows its textbook definition and returns a complete trace, so that any result can be checked by hand. Second, the simulators are validated against known answers and against theoretical properties. Third, the simulators are run on seeded random workloads to measure how the algorithms compare, and the optimizer turns those comparisons into recommendations. Every random workload comes from a numbered seed, so every number in Chapter 6 can be reproduced by running `bench.py`.

5.2 CPU scheduling

A process has an arrival time, a CPU burst and, for priority scheduling, a priority in which a smaller number means more urgent. Time advances in whole units. For every process the simulator records the time of its first run and of its completion, and from them the standard measures: **turnaround time** = completion − arrival; **waiting time** = turnaround − burst; **response time** = first run − arrival. The system measures are **utilisation** (busy time divided by the time from the first arrival to the last completion), **throughput** (processes completed per time unit) and the number of **context switches**, counted whenever the processor passes from one process to a different one.

Table 5.1: CPU scheduling algorithms

Algorithm	Process chosen	Pre-emptive	Strength	Weakness
FCFS	earliest arrival	no	simple, fair by arrival	convoy effect: short jobs wait behind a long one
SJF	shortest burst	no	low average waiting	needs the burst in advance; starves long jobs
SRTF	shortest remaining time	yes	minimum average waiting time	starves long jobs; more switches
PRIORITY	smallest priority number	no	respects importance	low-priority starvation; ignores length
PRIORITY-P	smallest priority number, at once	yes	urgent work starts immediately	same starvation risk
RR	queue order, one quantum at a time	yes	quick first response, fair	many switches; waiting grows with the quantum

Ties are broken by arrival time and then by the order in which processes were entered, and a running process is kept when it ties with a newcomer, which avoids context switches that would change nothing. The non-pre-emptive algorithms run a process to the end of its burst. The pre-emptive algorithms re-decide at every time unit. Round robin keeps a queue:

a process runs for at most one quantum, processes that arrive during that time join the queue before the interrupted process is put at its end (the convention of the textbooks), and when the queue is empty the processor idles until the next arrival. Idle periods appear in the Gantt chart and lower the utilisation.

For average waiting time SRTF is not merely good but provably optimal among all scheduling disciplines that do not idle while work is waiting [6]. The argument is an exchange: if a process with more remaining work is ever run while a shorter one is waiting, swapping them cannot increase the sum of the completion times. This gives a property that is easy to test: no other algorithm may beat SRTF on any workload (Section 5.7).

5.3 Page replacement

A reference string is the sequence of pages that a program touches. Memory has a fixed number of frames. A reference to a page that is already in a frame is a hit; otherwise a page fault occurs, and if all frames are full a victim must be chosen. The four rules are: **FIFO** removes the page that has been in memory longest. **LRU** removes the page that has not been used for the longest time, which uses the recent past to predict the near future. **OPT** removes the page whose next use is farthest away, which gives the fewest possible faults for the string [10] but cannot be used in a real system because it needs the future; it serves here as a lower bound. **CLOCK** (second chance) [4] arranges the frames in a circle with a reference bit per page and a hand: the hand skips pages whose bit is set, clearing the bit as it passes, and replaces the first page whose bit is clear, so a page that was used again gets a second chance. CLOCK approximates LRU at a much lower cost.

Two classical properties are tested. A replacement rule is a **stack algorithm** if the set of pages in memory with n frames is always contained in the set with $n + 1$ frames; LRU and OPT are stack algorithms [8], so adding a frame can never add faults. FIFO is not, and it can suffer **Belady's anomaly** [5]: on the string 1 2 3 4 1 2 5 1 2 3 4 5 it makes 9 faults with three frames and 10 with four.

5.4 Disk scheduling

The disk has a number of cylinders (200 in the experiments), the head is on one of them, and a list of cylinder numbers is waiting. **FCFS** serves them in arrival order. **SSTF** always serves the nearest request. **SCAN** [7], the elevator algorithm, sweeps in one direction to the end of the disk, serving requests on the way, then reverses. **C-SCAN** sweeps in one direction only: at the end it returns to the other end and sweeps again, which gives more

uniform waiting. **LOOK** and **C-LOOK** are the versions of SCAN and C-SCAN that turn round at the last request instead of at the end of the disk. The total head movement is the sum of the distances travelled; the return trip of C-SCAN and C-LOOK is counted because the arm really moves that distance. Table 5.2 gives the totals for the textbook queue 98, 183, 37, 122, 14, 124, 65, 67 with the head at cylinder 53, which can be checked by hand.

Table 5.2: Head movement for the textbook queue (head at 53, 200 cylinders)

Algorithm	Moving up first	Moving down first
FCFS	640	640
SSTF	236	236
SCAN	331	236
C-SCAN	382	386
LOOK	299	208
C-LOOK	322	326

5.5 The optimizer

The optimizer is deliberately simple, so that its decisions can be explained. For each resource it runs all algorithms on the same workload and ranks them by one metric: average waiting time for the CPU, number of page faults for memory (OPT excluded from the choice), and total head movement for the disk. Ties are broken by the order of the algorithm list. The saving is the percentage by which the best result is lower than the baseline, and a short explanation of the winner, with its known drawback, is attached to the answer. The other metrics are always reported as well, because the best algorithm for waiting time is not always the best for response time or fairness.

Quantum tuning. For round robin the optimizer sweeps the quantum and scores each value with *average turnaround* + *c* × *context switches* ÷ *n*, where *n* is the number of processes and *c* is the time lost at each context switch. The value of *c* is a parameter of the model (0.5 time units in the experiments), not a measurement. A small quantum gives a fast response but many switches; a large quantum approaches FCFS. **Frame tuning.** For memory the optimizer computes the fault rate for 1, 2, 3, ... frames and reports the smallest number that reaches a target rate. **Combined scenario.** The function `optimise()` takes one workload for each resource, applies the three recommendations and reports the saving of each against its baseline.

5.6 Workload generation

Table 5.3: Generators of workloads (all seeded)

Workload	Parameters
Processes (mixed)	8 processes; arrival uniform from 0 to 30; 70 % short bursts (1 to 5) and 30 % long bursts (10 to 30); priority 1 to 5
Processes (short)	same, with every burst between 1 and 6
Page references (locality)	200 references over 20 pages; the program works in a set of 5 pages for 40 references (90 % of references fall in the set, 10 % anywhere), then moves to a new set
Page references (uniform)	200 references, every one of 20 pages equally likely: no locality at all
Disk requests (uniform)	20 requests on 200 cylinders, head at a random cylinder
Disk requests (clustered)	20 requests around three busy regions (normal distribution, standard deviation 8 cylinders)

The mixed process workload imitates a system with many short interactive jobs and a few long computations. The locality reference string imitates the way real programs reuse a working set of pages for a while and then change phase [9]. The uniform string is the worst case for any rule that tries to predict the future from the past.

5.7 Testing methodology

The 29 tests are of four kinds. **Textbook cross-checks** reproduce the worked examples of [1]: average waiting 17, 3 and 5.67 for FCFS, SJF and round robin; 15, 12 and 9 faults for FIFO, LRU and OPT; the totals of Table 5.2. **Hand-checked small cases** test pre-emption (SRTF and PRIORITY-P interrupt a running process, SJF does not), idle gaps, the second chance of CLOCK and the frame table. **Property tests** run on generated workloads: SRTF is never beaten on average waiting time (150 workloads); the faults of LRU and OPT never increase when frames are added (60 strings); OPT is never beaten by any rule (40 strings); LOOK never moves more than SCAN (50 queues); every request is visited by every disk algorithm; and for every process of every algorithm waiting = turnaround − burst, while the total busy time equals the total of the bursts (360 simulations). **Interface tests** call the command line and the three HTTP endpoints and check that bad input returns an error instead of a wrong answer.

Table 5.4: Cost of the simulations (n = processes or references, F = frames, T = total CPU time, R = requests)

Simulator	Time	Remark
CPU, non-pre-emptive and pre-emptive	$O(T \cdot n)$	one decision per time unit for the pre-emptive algorithms
CPU, round robin	$O(n + T / q)$	one decision per quantum
Page replacement, FIFO, LRU, CLOCK	$O(n \cdot F)$	scan of the frames at a fault
Page replacement, OPT	$O(n^2 \cdot F)$ worst case	looks forward in the string for every candidate
Disk, SSTF	$O(R^2)$	the nearest request is searched again at every step
Disk, the others	$O(R \log R)$	sorting the requests

6. RESULTS

6.1 Test results

All 29 automated tests pass (`python -m unittest discover -s tests`, run time under one second). Table 6.1 lists them by group.

Table 6.1: Automated tests

Group	Tests	What is checked
CPU scheduling	9	textbook waiting times; round-robin Gantt chart; SRTF pre-empts and SJF does not; priority order; pre-emptive priority; idle gap and utilisation; time accounting on 360 simulations; SRTF never beaten on 150 workloads; bad input
Page replacement	7	15, 12, 9 faults; Belady's anomaly for FIFO and not for LRU; no anomaly for LRU and OPT on 60 strings; OPT lower bound on 40 strings; only cold faults with enough frames; CLOCK second chance; frame table
Disk scheduling	5	totals in both directions; every request visited; LOOK never above SCAN on 50 queues; bad input
Optimizer	6	CPU, memory and disk recommendations; quantum sweep; frames needed; combined scenario has no negative saving
Interfaces	2	command line; HTTP endpoints, including the 400 reply for a request outside the disk

6.2 Textbook examples reproduced

The first check of a simulator is that it gives the answers printed in the textbook. Three processes arrive at time 0 with bursts of 24, 3 and 3. Figure 6.1 shows the Gantt charts. FCFS runs the long process first, so the two short ones wait 24 and 27 units and the average waiting time is 17. SJF runs the short ones first and the average falls to 3, which is 82% lower. Round robin with a quantum of 4 gives 5.67 (6, 4 and 7 units). These are the numbers of [1]. For page replacement the reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 with three frames gives 15 faults for FIFO, 12 for LRU and 9 for OPT, and 14 for CLOCK. Figure 6.2 shows how the arm moves for four disk algorithms on the queue of Table 5.2.

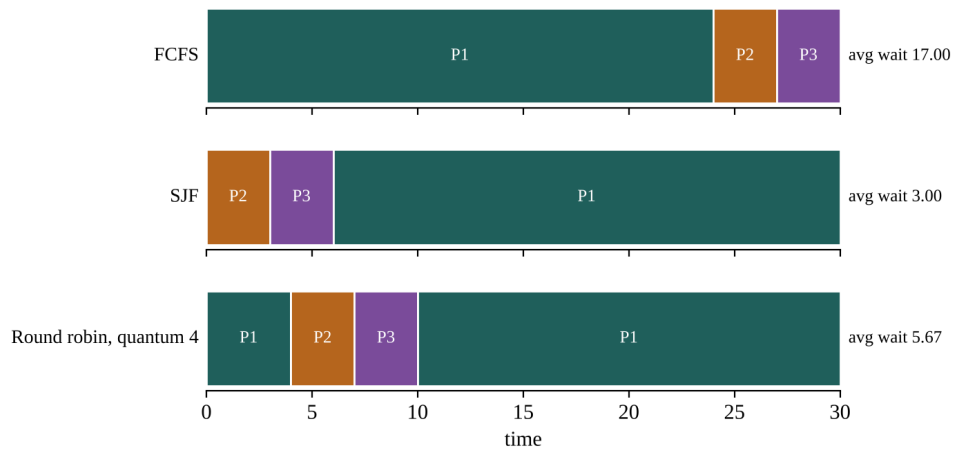


Figure 6.1: Gantt charts of FCFS, SJF and round robin for the textbook example

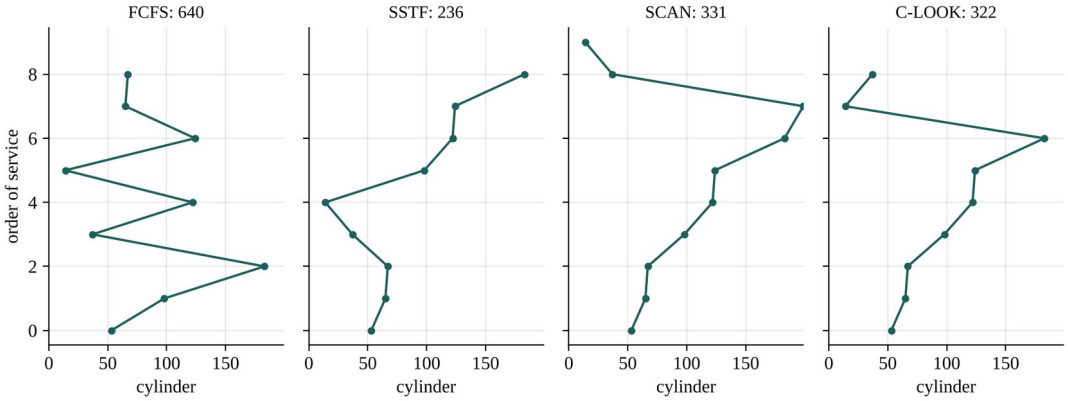


Figure 6.2: Head movement for the textbook queue (head at 53, moving up first); the total is in each title

6.3 CPU scheduling on 500 generated workloads

Table 6.2 gives the average of each measure over 500 mixed workloads of eight processes (quantum 4), and Figure 6.3 shows the waiting and response times. The last column is the share of the workloads in which the algorithm reached the lowest average waiting time (ties count for every algorithm involved).

Table 6.2: CPU scheduling, mean of 500 mixed workloads

Algorithm	Avg waiting	Avg turnaround	Avg response	Max waiting	Switches	Best
FCFS	18.15	26.38	18.15	37.86	7.0	1.6%
SJF	11.3	19.53	11.3	32.5	7.0	11.2%
SRTF	7.69	15.92	5.93	31.53	8.95	100.0%
PRIORITY	18.38	26.61	18.38	43.95	7.0	1.8%
PRIORITY-P	17.8	26.03	14.79	44.93	8.47	1.8%
RR	16.14	24.37	5.13	32.69	17.1	0.8%

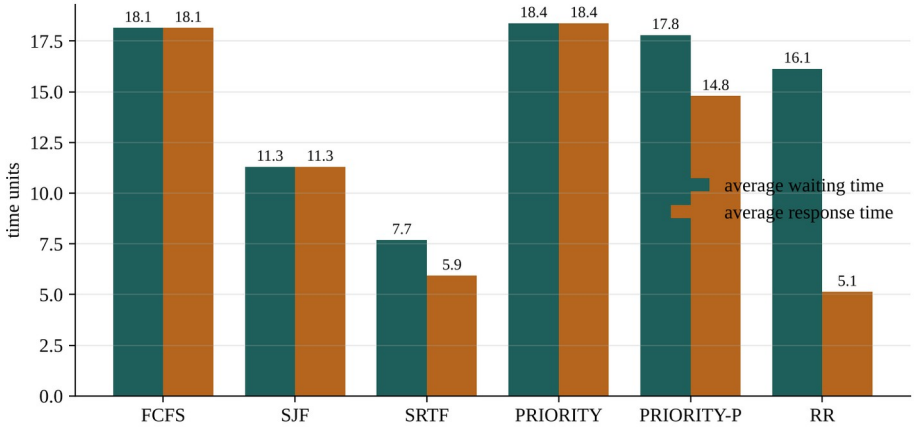


Figure 6.3: Average waiting and response time of the six algorithms

SRTF has the lowest average waiting time (7.69) and was best or tied-best in 100% of the workloads, as the theory predicts; it is 58% lower than FCFS (18.15). Non-pre-emptive SJF reaches 11.3, so about two thirds of SRTF's gain over FCFS comes from simply choosing the short job, and the rest from pre-empting. Priority scheduling gives no gain here (18.38 and 17.8) because the priorities in the generator are random and unrelated to the length of the jobs. Round robin has the best response time (5.13, against 5.93 for SRTF) but needs 17.1 context switches per workload, about twice as many as the others, and its average waiting time (16.14) is much worse than SRTF's.

Two further observations are worth recording. **Utilisation is the same for all six algorithms** (0.956) because every one of them keeps the processor busy whenever work is waiting; the idle time comes only from gaps between arrivals. A scheduler cannot raise utilisation, it can only choose whose work is done first, so the optimisation of the CPU is about waiting and response time. And the price of SRTF is fairness: its average maximum waiting time (31.53) is only 17% below FCFS's (37.86), so the longest-waiting job gains little while the short ones gain a lot. For a workload of only short jobs round robin falls behind even FCFS on waiting time: SRTF 2.24, FCFS 3.12, RR 3.9, because a quantum of 4 cuts jobs that would have finished in one turn.

6.4 Choosing the time quantum

Figure 6.4 sweeps the round-robin quantum from 1 to 20 on 300 mixed workloads. The average response time grows steadily with the quantum (from 1.27 at q = 1 to 16.05 at q = 20) and the number of context switches falls from 55.21 to 8.08. The average waiting time does not move in a straight line: it is lowest at q = 5 (15.68) and higher on both sides (16.43 at q = 4, 19.77 at q = 20). The reason is in the generator: 70% of the jobs have

bursts of 1 to 5 units. With q = 5 every short job finishes in one turn and never queues a second time, while a smaller quantum splits it. This is the rule of thumb of the textbooks, that most CPU bursts should be shorter than the quantum, observed on a generated workload. With a context-switch cost of 0.5 time units the combined score of Section 5.5 is also lowest at q = 5 (24.88), so the optimizer recommends q = 5 for this kind of workload. A system with different job lengths would get a different answer, which is why the tool sweeps instead of fixing a number.

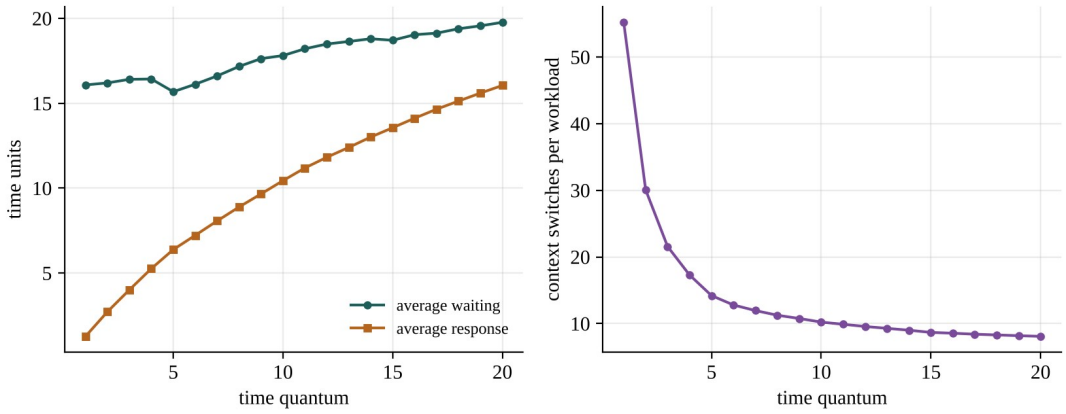


Figure 6.4: Effect of the time quantum on waiting, response and context switches

6.5 Page replacement and the number of frames

Table 6.3: Page-fault rate (%), mean of 200 reference strings with locality

Frames	FIFO	CLOCK	LRU	OPT
3	52.8	52.5	51.9	35.2
4	40.1	39.4	38.0	24.6
5	30.3	28.7	26.3	18.2
6	24.1	22.2	20.0	15.0
8	17.8	16.4	15.5	11.9
10	15.2	14.0	13.7	10.4
12	13.1	12.3	12.1	9.5

Figure 6.5 and Table 6.3 show the fault rate of the four rules on 200 reference strings with locality. The ranking is the same at every number of frames: OPT is lowest, then LRU, CLOCK and FIFO. With five frames the fault rates are 30.3% for FIFO, 28.7% for CLOCK, 26.3% for LRU and 18.2% for OPT, so LRU removes 13% of the faults of FIFO, and CLOCK, which needs only one bit per page, recovers 38% of that gain. Adding frames helps with diminishing returns: to bring the fault rate down to 18% FIFO needs 8 frames, CLOCK 8 and LRU 7, so a better replacement rule is worth a frame of memory. The gap between LRU and OPT never closes completely; it is the price of not knowing the future.

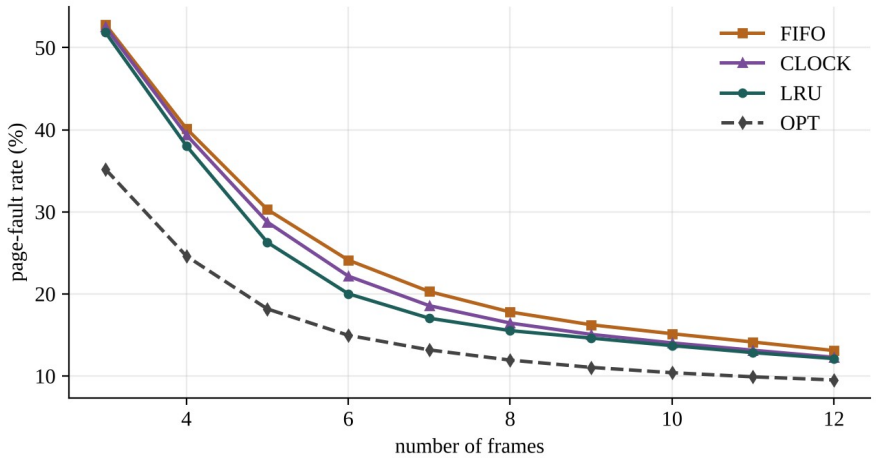


Figure 6.5: Page-fault rate against the number of frames

Locality is what makes LRU useful. On strings with no locality (every page equally likely) with 8 frames the three real rules are indistinguishable: FIFO 60.9%, LRU 61.1%, CLOCK 61.2%, while OPT, which sees the future, achieves 35.4%. When the past says nothing about the future, no rule that learns from the past can do better than chance, and the optimizer reports that LRU's advantage in Table 6.3 belongs to the workload and not to the algorithm alone.

6.6 Belady's anomaly

On the textbook string 1 2 3 4 1 2 5 1 2 3 4 5 FIFO makes 9 faults with three frames and 10 with four (Figure 6.6): more memory, more faults. CLOCK shows the same behaviour on this string, while LRU improves from 10 to 8 and OPT from 7 to 6. The anomaly is not only a textbook curiosity. In 500 random strings of 60 references over 8 pages, FIFO showed it in 4 strings and CLOCK in 1, while LRU and OPT showed it in none, as they must since they are stack algorithms. It is rare, but it is a reason why the optimizer never treats "more frames" as automatically better for FIFO.

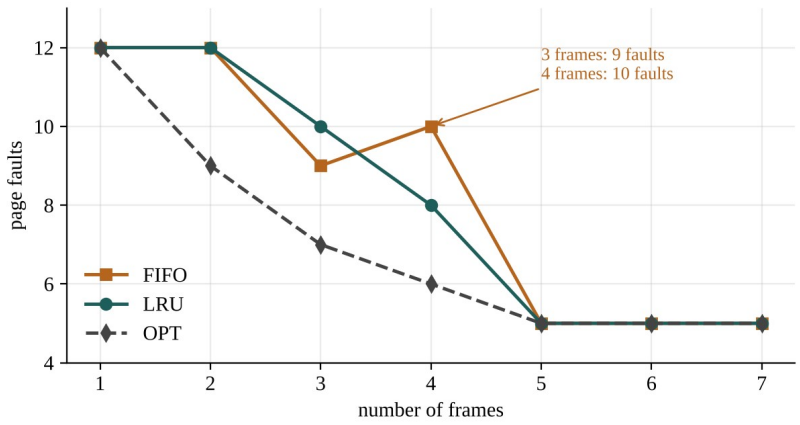


Figure 6.6: Belady's anomaly on the textbook string

6.7 Disk scheduling

Table 6.4: Head movement per queue of 20 requests (mean of 300 queues)

Algorithm	Uniform	Less than FCFS	Clustered	Less than FCFS
FCFS	1347.2	0%	958.8	0%
SSTF	257.9	81%	162.5	83%
SCAN	279.6	79%	232.9	76%
C-SCAN	375.0	72%	326.4	66%
LOOK	263.1	80%	176.7	82%
C-LOOK	343.2	75%	223.9	77%

Table 6.4 and Figure 6.7 compare the six algorithms on 300 queues of 20 requests. FCFS moves the arm about 1347 cylinders for a queue of uniformly spread requests, SSTF only 258, and LOOK 263: serving the requests in an order that follows the geometry of the disk removes more than 80% of the movement. SSTF is the shortest, but only slightly (258 against 263 for LOOK), and it can starve a request that is far from a busy region, which LOOK cannot because it sweeps the whole disk. SCAN and C-SCAN pay for travelling to the ends (280 and 375); C-LOOK (343) gives the most uniform waiting among the sweep algorithms at the cost of the return trip. When the requests are clustered the differences between the pure sweep algorithms grow: SSTF 162, LOOK 177, C-LOOK 224, SCAN 233, because the long trips to the end of the disk are wasted.

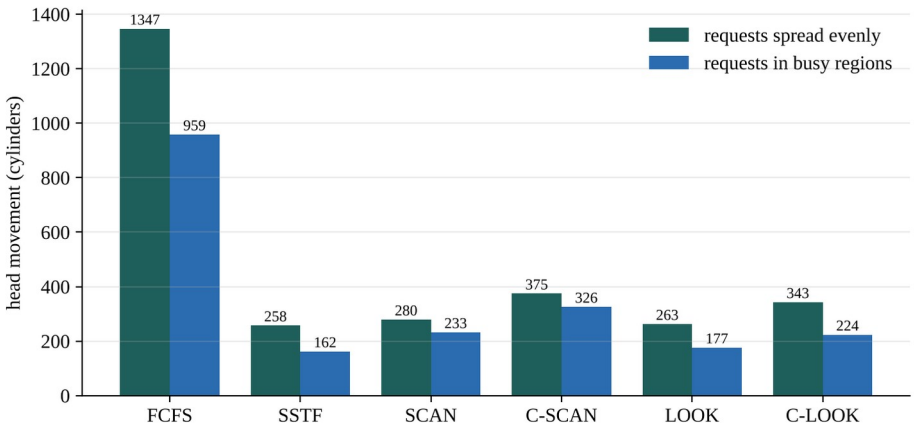


Figure 6.7: Average head movement of the six algorithms

6.8 One scenario, three resources

To show the optimizer working on the whole system, one scenario was generated from seed 11: eight processes, a reference string of 200 pages with five frames, and 20 disk requests. Table 6.5 shows the baseline, the recommended choice and the saving for each resource.

Table 6.5: Combined scenario (seed 11)

Resource	Measure	Baseline	Recommended	Result	Saving
CPU	average waiting time	FCFS: 18.0	SRTF	6.62	63.2%
Memory	page faults (5 frames)	FIFO: 67	LRU	58	13.4%
I/O	head movement (cylinders)	FCFS: 934	LOOK	198	78.8%

The CPU and the disk gain most (63% and 79%) because FCFS ignores everything about the work, while memory gains 13% because even FIFO keeps some useful pages. These are the results for one scenario, not a promise: Tables 6.2 to 6.4 give the typical gains over many workloads.

6.9 Browser interface

Figures 6.8 to 6.10 show the three tabs of the browser interface (`python -m resopt.server`). Each tab has an input area filled with an example, a button that runs all algorithms, a comparison table in which the recommended row is highlighted, and a picture of the result.

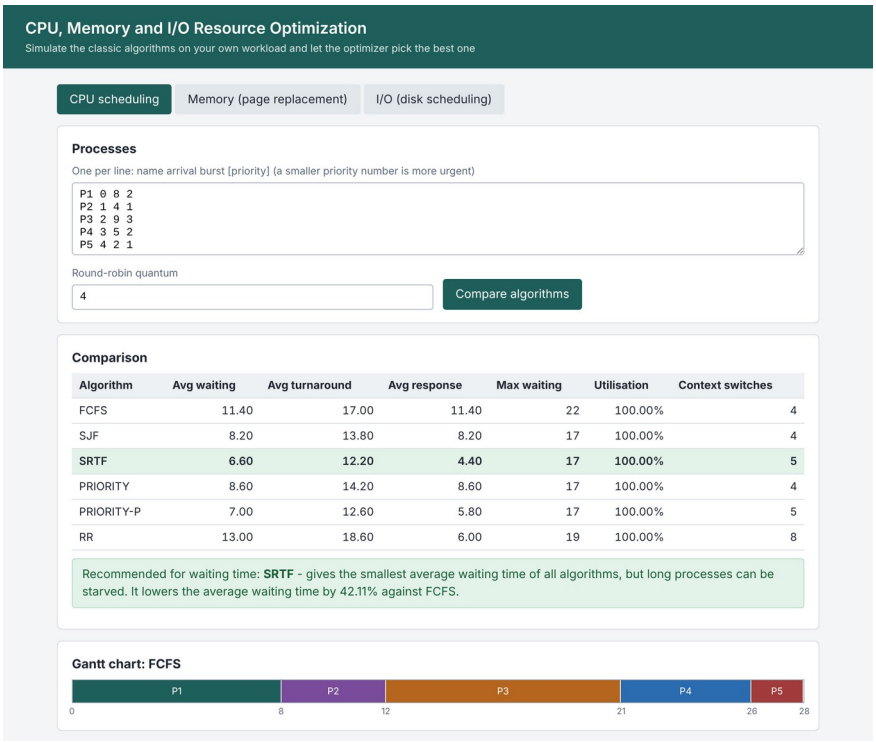


Figure 6.8: CPU tab: comparison table, recommendation and Gantt chart

In Figure 6.8 five processes with their priorities are compared. SRTF is highlighted: its average waiting time of 6.60 is 42.11% lower than the 11.40 of FCFS, and the explanation notes that long processes can be starved. The Gantt chart of FCFS runs the processes in order of arrival, while that of SRTF shows P1 pre-empted at time 1 by the shorter P2 and resumed at time 12.

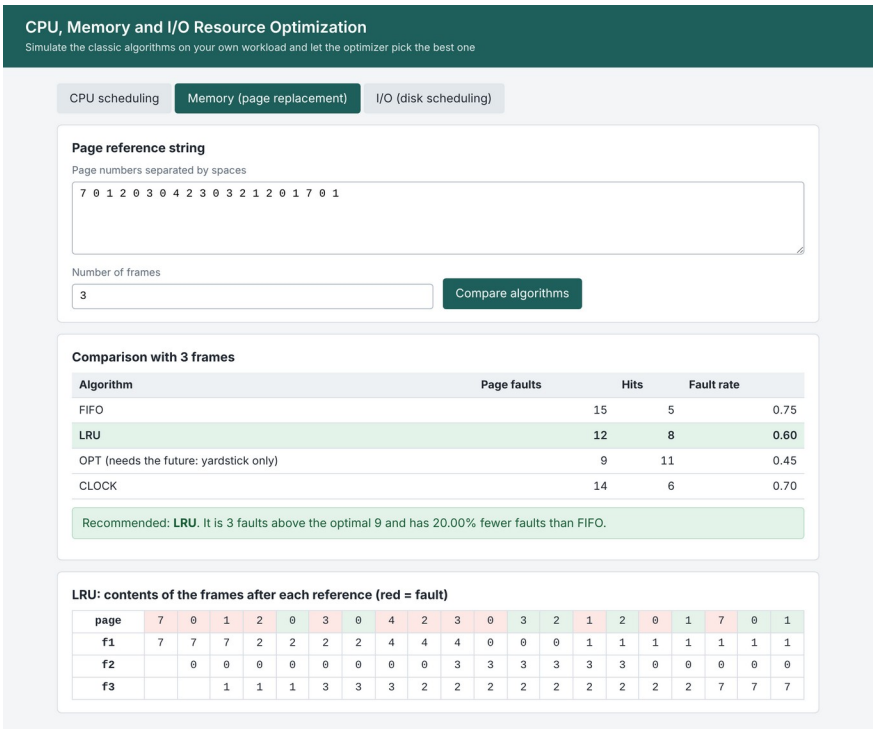


Figure 6.9: Memory tab: faults of each rule and the frame table of the recommended one

Figure 6.9 repeats the textbook string with three frames. LRU is recommended with 12 faults, 3 above the optimal 9, and 20% fewer faults than FIFO. The frame table below shows, for LRU, the page referenced at each step (red where a fault occurred) and the content of each frame after the reference.

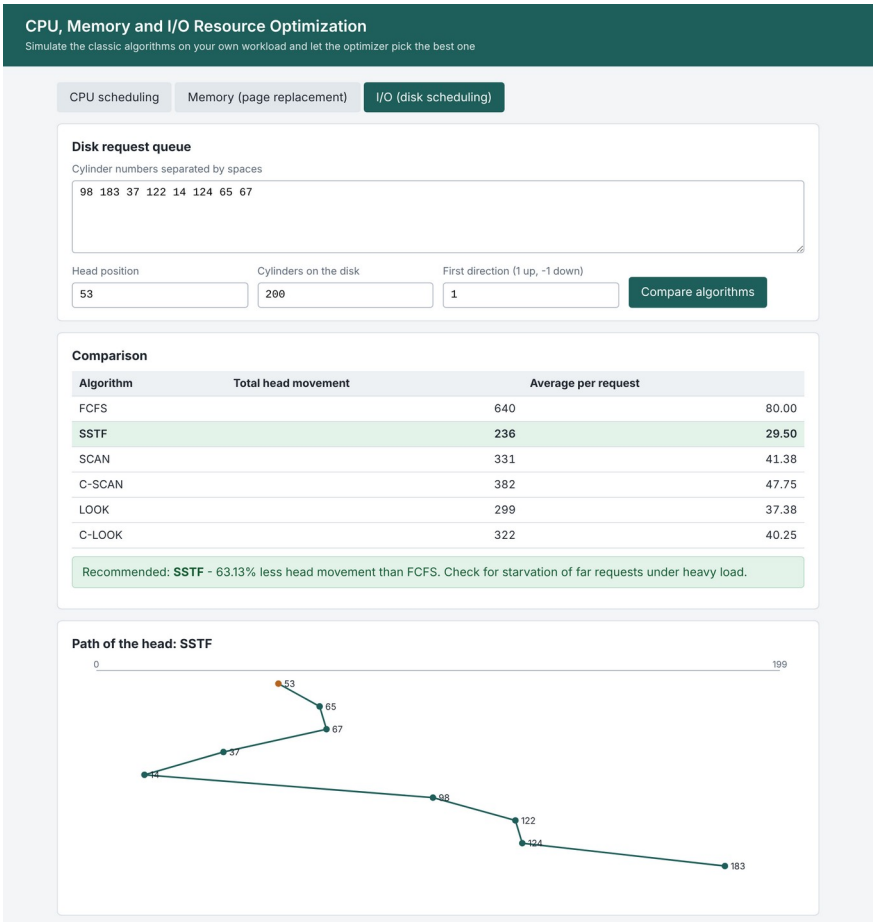


Figure 6.10: I/O tab: head movement of each algorithm and the path of the recommended one

Figure 6.10 shows the disk tab on the textbook queue with the head at 53 and the arm moving up first. SSTF is recommended with 236 cylinders, 63.13% less than FCFS's 640, and the path of the head is drawn with the starting cylinder in orange. The reminder under the table, to check for starvation, appears because the shortest-seek rule can delay far requests.

6.10 Performance

Table 6.6: Time to run all algorithms of one resource

Workload	Time (ms)
CPU, 50 processes (all 6 algorithms)	6.49
CPU, 100 processes (all 6 algorithms)	20.6
CPU, 200 processes (all 6 algorithms)	79.06
Memory, 1000 references, 8 frames (all 4 algorithms)	5.14
Memory, 5000 references, 8 frames (all 4 algorithms)	30.86
Disk, 1000 requests (all 6 algorithms)	41.32

The simulators are fast enough for interactive use: all six CPU algorithms on 200 processes take 79 ms, and the four page-replacement rules on 5,000 references take about 31 ms. OPT dominates the memory time because it looks ahead in the string, and the pre-emptive

CPU algorithms dominate the CPU time because they decide at every time unit. The times are from one machine and one Python version and show the order of magnitude only.

6.11 Discussion

The results support the aims of the project. The simulators agree with the textbook, and with each other on the properties that theory demands: SRTF is never beaten for waiting time, LRU and OPT never show Belady's anomaly, OPT is a lower bound, LOOK never exceeds SCAN. On generated workloads the choice of algorithm matters a great deal for waiting time and head movement and less for page faults, and the best algorithm depends on the workload: round robin wins on response time and loses on waiting, LRU wins only when the program has locality, and SSTF wins on movement but not on fairness. A tool that reports only one metric would hide these trade-offs; the optimizer reports all of them and recommends on the main one.

The experiments also have limits. The workloads are synthetic and their parameters (70% short jobs, a locality set of five pages) were chosen by the authors, so the absolute numbers would change for other systems; only the orderings that follow from theory, and the way each trade-off appears, can be expected to carry over. The CPU model gives no cost to a context switch and the disk model ignores rotational delay, which favours the algorithms that switch or move a lot. Real operating systems also combine these ideas, for example multilevel feedback queues and elevator-style disk schedulers with deadlines, which are beyond this study.

7. CONCLUSION

This mini project built a simulator and optimizer for the three resources that an operating system shares: the processor, the memory and the disk. Six CPU scheduling algorithms, four page-replacement rules and six disk-scheduling algorithms were implemented from their definitions, compared on the same workloads, and tied together by an optimizer that ranks them, recommends one with its reason, and tunes the round-robin quantum and the number of frames. The simulators reproduce the textbook examples exactly, and 29 automated tests, including property tests on generated workloads, pass.

The experiments show how large and how workload-dependent the differences are. SRTF lowered the average waiting time by 58% against FCFS and was never beaten; the best round-robin quantum was 5 because most jobs were that short; LRU removed 13% of FIFO's faults on programs with locality and nothing on programs without it; and LOOK or SSTF removed more than 80% of the head movement of FCFS. No algorithm was best on every measure, which is the reason for an optimizer that reports all of them. Choosing the right policy for the workload, and not adding hardware, is the cheapest way to make a system faster.

7.1 Limitations

- The tool is a simulator: it models time in whole units and does not run inside a real kernel.
- Bursts are known in advance, which real schedulers must estimate; processes have no I/O phases of their own.
- A context switch costs no time unless a cost is given to the tuning function, and the disk model ignores rotational delay.
- Page replacement is studied for one reference string against a fixed number of frames, not for several processes competing for memory.
- All disk requests are present at the start; requests that arrive while the arm is moving are not modelled.
- The workloads are synthetic with parameters chosen by the authors, so absolute numbers depend on them.

7.2 Future scope

- Add multilevel feedback queues, highest-response-ratio-next and real-time schedulers (rate monotonic, earliest deadline first).

- Estimate burst lengths by exponential averaging so that SJF and SRTF can be used without knowing the future.
- Add process I/O phases, so that CPU, memory and disk are simulated together in one timeline.
- Model several processes with working-set or page-fault-frequency allocation, and thrashing.
- Replay real traces (for example the page references of a real program or a disk trace) instead of generated workloads.
- Add arrival over time for disk requests and starvation measures such as the longest wait.
- Report confidence intervals for the generated experiments and allow the user to save and reload workloads.

8. REFERENCE

[1] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.

[2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Boston, MA: Pearson, 2014.

[3] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. New York, NY: Pearson, 2018.

[4] F. J. Corbató, "A paging experiment with the Multics system," in In Honor of Philip M. Morse, H. Feshbach and K. U. Ingard, Eds. Cambridge, MA: MIT Press, 1969, pp. 217–228.

[5] L. A. Belady, R. A. Nelson and G. S. Shedler, "An anomaly in space-time characteristics of certain programs running in a paging machine," Communications of the ACM, vol. 12, no. 6, pp. 349–353, 1969.

[6] L. Schrage, "A proof of the optimality of the shortest remaining processing time discipline," Operations Research, vol. 16, no. 3, pp. 687–690, 1968.

[7] P. J. Denning, "Effects of scheduling on file memory operations," in Proceedings of the AFIPS Spring Joint Computer Conference, 1967, pp. 9–21.

[8] R. L. Mattson, J. Gecsei, D. R. Slutz and I. L. Traiger, "Evaluation techniques for storage hierarchies," IBM Systems Journal, vol. 9, no. 2, pp. 78–117, 1970.

[9] P. J. Denning, "The working set model for program behavior," Communications of the ACM, vol. 11, no. 5, pp. 323–333, 1968.

[10] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," IBM Systems Journal, vol. 5, no. 2, pp. 78–101, 1966.